

Cifrari a blocchi ed Encryption modes: CBC, ECB, OFB e CTR

Domande:

1. CBC (x10)
2. ECB (x11)
3. differenze CBC/ECB (x10)
4. CBC and error propagation (x10)
5. Argue whether CBC can be used with public key encryption schemes (x10)
6. schema modalità CBC (x10)
7. Block cipher: per costruire cifrario a caratteri, modalità -> OFB, CFB, CTR; (dimostrazione che OFB non può essere un cifrario perfetto NON SPIEGATO) (x10)
8. Block cipher: costruzione cifrario perfetto, mostrare che non è possibile in pratica (permutazioni...); che permutazione implementa un cifrario? (x10)
9. Da Block Cipher a stream cipher, Encryption modes (x10)

Risposte:

Cifratura simmetrica e cifrari a blocchi

Per definizione un cifrario simmetrico è una coppia di funzioni del tipo:

$$E : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$$

$$D : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$$

Prendono in entrata un plaintext da n bit e una chiave da k bit, restituiscono un crittogramma da n bit.

Cifrario a blocchi perfetto (true random cipher)

Dalla definizione di cifrario a blocchi si deduce che $|M| = |C|$ e quindi la cifratura è di fatto una permutazione.

Il set di permutazioni che lega ogni plaintext al relativo crittogramma è data dalla chiave.

Un cifrario a blocchi è perfetto solo se implementa **tutte** le possibili permutazioni: ossia $(2^n!)$ per avere ciò serve che i bit di $k \sim \log_2 2^n! \sim (n - 1,44) * 2^n$

Quindi la perfezione implica avere una chiave molto grande.

Dictionary attack

Tipologia di Known Plaintext Attack.

L'avversario conserva una tabella T composta da molte coppie $\langle p, c \rangle$, l'avversario ovviamente non conosce k e non conosce nemmeno come funziona il cifrario, ha solo molte corrispondenze $c \leftrightarrow p$.

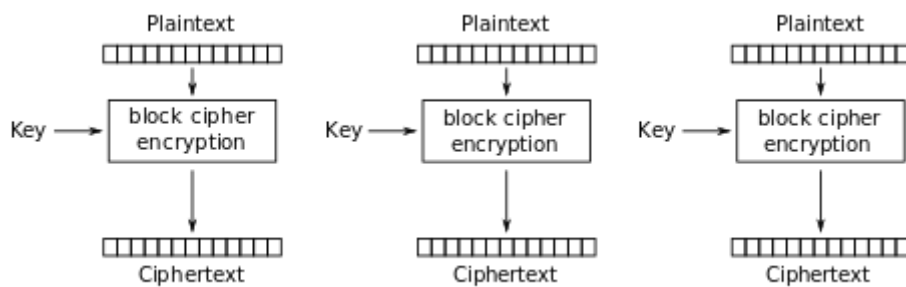
Nel caso ideale, l'avversario ha tutte le possibili coppie, quindi 2^n coppie.

1. L'avversario intercetta un crittogramma c'
2. Se nella tabella T esiste una coppia con c' , allora il testo corrispondente è p' .

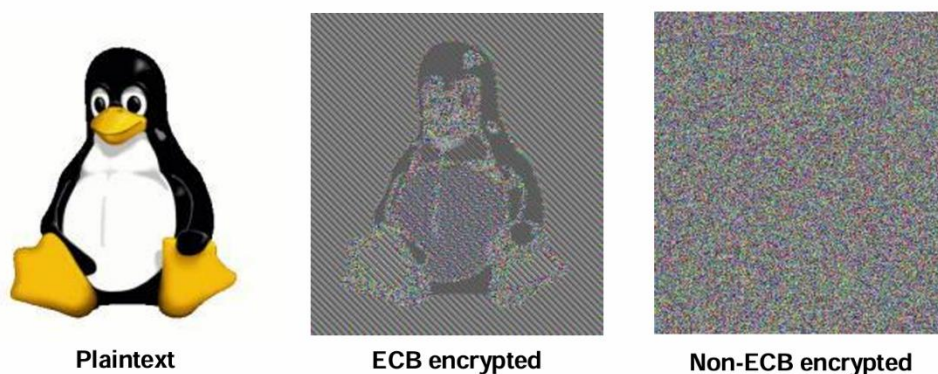
Questo attacco è molto potente ma è possibile solo se:

- La chiave non viene mai cambiata, ad ogni cambio la tabella diventa inutile perché le permutazioni sono cambiate.
- Se il numero di coppie possibili è piccolo, altrimenti diventa infattibile; ovvero se n è piccolo.

ECB - Electronic CodeBook



La modalità ECB (Electronic Codebook) è la più semplice tra le modalità di cifratura per cifrari a blocchi. In questa modalità, ogni blocco di testo in chiaro viene cifrato indipendentemente utilizzando la stessa chiave. Ciò significa che blocchi identici di testo in chiaro verranno cifrati in blocchi identici di testo cifrato.



Vantaggi:

- Non è richiesta la sincronizzazione dei blocchi.
- Non c'è propagazione degli errori.
- Uno o più bit in un singolo blocco cifrato influenzano solo la decifratura di quel blocco.
- Può essere parallelizzato.

Svantaggi (è insicuro):

- Testi in chiaro identici risultano in testi cifrati identici.
- ECB non nasconde i pattern dei dati.
- Permette l'analisi del traffico.
- I blocchi sono cifrati separatamente.
- Consente il riordino e la sostituzione dei blocchi.

Attacco a blocchi in ECB

Un esempio di attacco a blocchi in modalità ECB può essere descritto attraverso una transazione bancaria:

- Una transazione bancaria trasferisce una somma di denaro D dal cliente C della banca B1 alla banca B2.
- La banca B1 addebita D a C e invia il messaggio "credito D a C" alla banca B2.
- La banca B2 riceve il messaggio e accredita D a C.

Formato del messaggio di credito:

- Banca di origine: M (12 byte)
- Banca destinataria: R (12 byte)

- Cliente: C (48 byte)
- Numero di conto bancario: N (16 byte)
- Somma di denaro: D (8 byte)

Attacco di Lou Cipher:

Scopo dell'attacco: Ripetere il messaggio "credito 100\$ a LouCipher" molte volte.

Strategia dell'attacco:

1. LouCipher attiva più trasferimenti di 100\$, facendo sì che vengano inviati più messaggi "credito 100\$ a LouCipher" da B1 a B2 criptati.
2. LouCipher identifica almeno uno di questi messaggi e lo replica più volte.

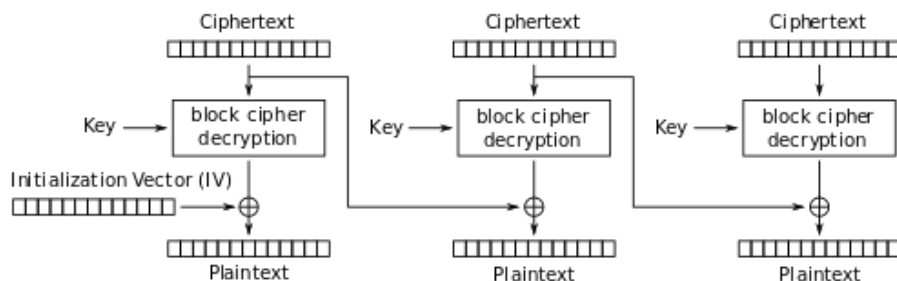
Contrasto dell'attacco:

- Il messaggio manca di un'indicazione temporale, quindi può essere facilmente replicato.
- Si aggiunge un campo timestamp di 8 byte (blocco #1) al messaggio per prevenire attacchi di replay.
- Un messaggio replicato può ora essere scartato.

Possibile continuazione dell'attacco:

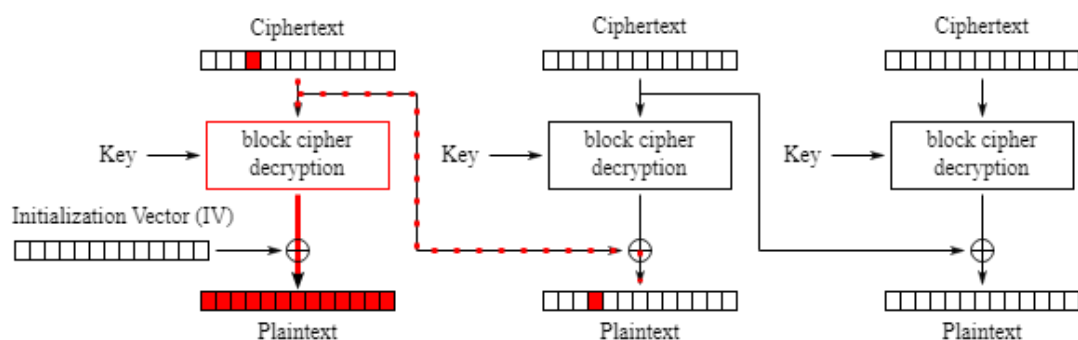
1. LouCipher identifica i "suoi" testi cifrati ispezionando i blocchi #2-#13.
2. Seleziona un qualsiasi suo testo cifrato.
3. Sostituisce il blocco #1 del suo testo cifrato con il blocco #1 di un qualsiasi blocco "fresco" intercettato.
4. Ripete il testo cifrato risultante.

CBC - Cipher Block Chaining



La modalità CBC (Cipher Block Chaining) è una tecnica di cifratura per cifrari a blocchi in cui ogni blocco di testo in chiaro viene combinato con il blocco cifrato precedente prima di essere cifrato. Questo metodo garantisce che ciascun blocco cifrato dipenda da tutti i blocchi di testo in chiaro precedenti, rendendo la cifratura più sicura.

Riguardo la propagazione dell'errore in decifratura. In CBC, se c'è un errore in un blocco cifrato durante la trasmissione o l'archiviazione, tale errore influenzerà non solo il blocco corrispondente durante la decifratura,

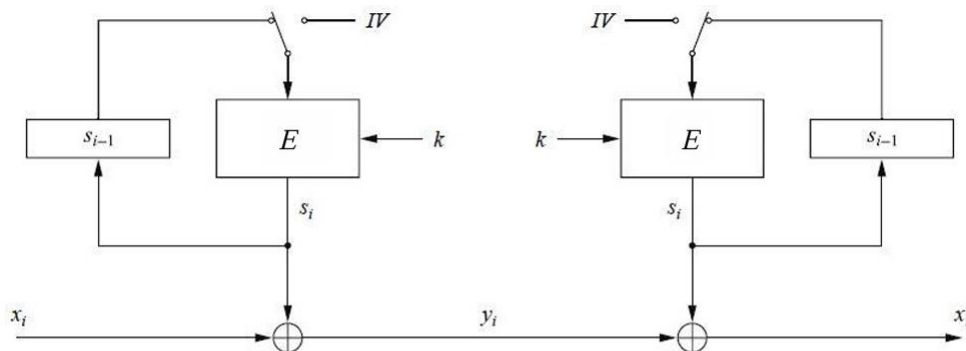


ma anche il blocco successivo. Questo accade perché ogni blocco di testo cifrato dipende dal blocco precedente.

CBC - Proprietà

- **Sicurezza:** La modalità CBC (Cipher Block Chaining) è sicura contro gli attacchi CPA (Chosen Plaintext Attack).
- **Dipendenza a catena:** Ogni blocco cifrato (c_i) dipende dal blocco di testo in chiaro (p_i) e dai blocchi di testo in chiaro precedenti.
- **Espansione del testo cifrato:** L'espansione del testo cifrato è di un solo blocco.
- **Randomizzazione:** La cifratura in modalità CBC è randomizzata utilizzando un IV (Initialization Vector) o nonce. Lo stesso testo cifrato risulta dallo stesso testo in chiaro sotto la stessa chiave e IV.
- **Riordinamento dei blocchi cifrati:** Il riordinamento dei blocchi cifrati influisce sulla decifratura.
- **IV (Initialization Vector):** Può essere inviato in chiaro, ma la sua integrità deve essere garantita.
- **Propagazione degli errori:** La modalità CBC soffre di propagazione degli errori. Gli errori di bit in (c_i) influenzano (p_i) e (p_{i+1}) .
- **Recupero dagli errori:** La modalità CBC è auto-sincronizzante per quanto riguarda il recupero dagli errori.
- **Tolleranza agli errori di framing:** La modalità CBC non tollera bit "persi" (errori di framing).
- **Parallelizzazione:** La decifratura in modalità CBC può essere parallelizzata.

Output Feedback Encryption (OFB)



Let $e()$ be a block cipher of block size b ; let x_i , y_i and s_i be bit strings of length b ; and IV be a nonce of length b .

Encryption (first block): $s_1 = e_k(IV)$ and $y_1 = s_1 \oplus x_1$

Encryption (general block): $s_i = e_k(s_{i-1})$ and $y_i = s_i \oplus x_i$, $i \geq 2$

Decryption (first block): $s_1 = e_k(IV)$ and $x_1 = s_1 \oplus y_1$

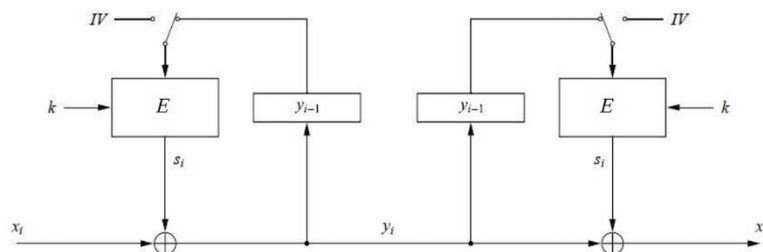
Decryption (general block): $s_i = e_k(s_{i-1})$ and $x_i = s_i \oplus y_i$, $i \geq 2$

- OFB crea un cifrario a flusso a partire da un cifrario a blocchi.
- Il flusso di chiavi è generato blocco per blocco.
- OFB è un cifrario a flusso sincrono: il flusso di chiavi è funzione solo della chiave (K) e del vettore di inizializzazione (IV). Questo permette la precomputazione del flusso di chiavi.
- Il ricevente non utilizza la decifratura.
- Se la lunghezza dell'ultimo blocco di testo in chiaro è inferiore alla dimensione del blocco, i bit del flusso di chiavi in eccesso vengono scartati.

Caratteristiche aggiuntive:

- Il IV deve essere un nonce (numero unico), rendendo OFB non deterministico.
- Non c'è propagazione degli errori.
- OFB soffre di malleabilità.

Cipher Feedback Encryption (CFB)



Definition 5.1.4 Cipher feedback mode (CFB)

Let $e()$ be a block cipher of block size b ; let x_i and y_i be bit strings of length b ; and IV be a nonce of length b .

Encryption (first block): $y_1 = e_k(IV) \oplus x_1$

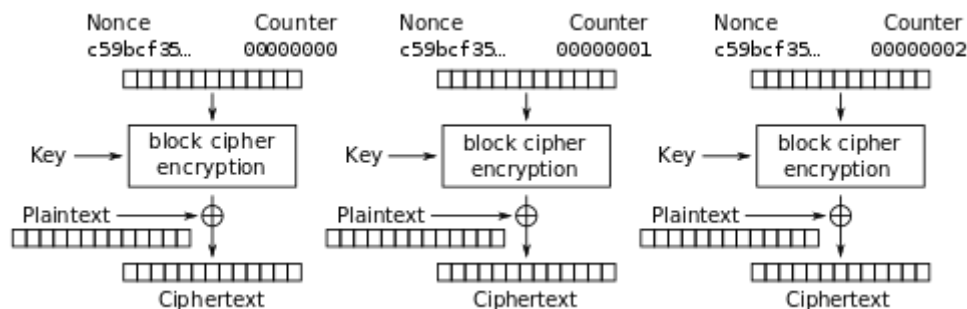
Encryption (general block): $y_i = e_k(y_{i-1}) \oplus x_i, \quad i \geq 2$

Decryption (first block): $x_1 = e_k(IV) \oplus y_1$

Decryption (general block): $x_i = e_k(y_{i-1}) \oplus y_i, \quad i \geq 2$

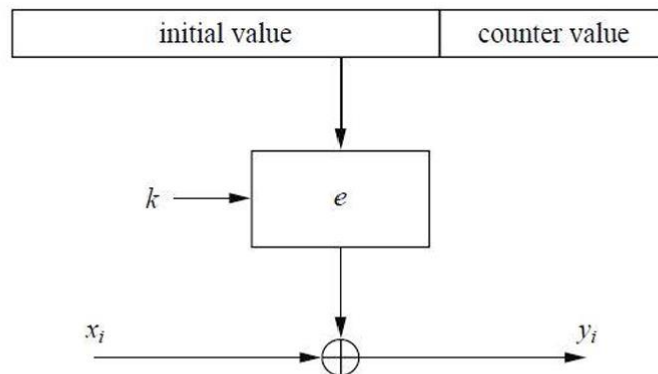
- **CFB crea un cifrario a flusso a partire da un cifrario a blocchi.**
- **CFB è un cifrario a flusso asincrono:** il flusso di chiavi è anche una funzione del testo cifrato (CT).
- Il flusso di chiavi è generato blocco per blocco.
- **IV è un nonce (numero unico), rendendo CFB non deterministico.**
- La cifratura (Enc) è sequenziale, mentre la decifratura (Dec) può essere parallelizzata.
- **CFB può operare su testi in chiaro (pt) e testi cifrati (ct) di dimensioni inferiori a un blocco:** Size of(pt/ct) = $s \leq n$ (cipher block size).

CTR - Counter Mode Encryption



I cifrari a blocchi possono essere trasformati in cifrari a flusso utilizzando diverse modalità operative. Le tre modalità principali sono OFB (Output Feedback), CFB (Cipher Feedback) e CTR (Counter). Ognuna di queste modalità ha caratteristiche uniche e differenti modalità di funzionamento.

- **CTR previene l'uso ripetuto del keystream (two-time pad).**
- **CTR può essere parallelizzato.**
- Il contatore può essere un contatore regolare o una funzione più complessa, come un registro a scorrimento con retroazione lineare (LFSR).



Definition 5.1.5 Counter mode (CTR)

Let $e()$ be a block cipher of block size b , and let x_i and y_i be bit strings of length b . The concatenation of the initialization value IV and the counter CTR_i is denoted by $(IV||CTR_i)$ and is a bit string of length b .

Encryption: $y_i = e_k(IV||CTR_i) \oplus x_i, \quad i \geq 1$

Decryption: $x_i = e_k(IV||CTR_i) \oplus y_i, \quad i \geq 1$

- **Espansione del testo cifrato:** È solo di un blocco.
- Output: $y_0, y_1, \dots, y_t$ con $y_0 = (IV|ctr_0)$ che è il blocco di espansione.
- $(IV|ctr_0)$ non deve essere tenuto segreto e può essere trasmesso insieme al testo cifrato.

Caratteristiche aggiuntive:

- **CTR è sicuro contro gli attacchi CPA (Chosen Plaintext Attack).**
- Un cifrario a blocchi è una buona approssimazione di una permutazione pseudo-casuale (PRP) o una funzione pseudo-casuale (PRF), quindi la sequenza $E_k(IV|ctr_0 + 1), \dots, E_k(IV|ctr_0+t)$ è pseudo-casuale.
- Il problema del two-time pad può verificarsi quando $(IV|ctr_0+i)$ si ripete, quindi è necessario limitare il numero massimo di messaggi che si possono cifrare.
- Il problema del two-time pad può verificarsi anche quando $(IV|ctr_0+i) = (IV'|ctr_0'+j)$, ma la probabilità che ciò accada è esponenzialmente piccola.

RSA, key-generation, Problema RSA-Plain + soluzioni, decryption e encryption

Domande:

1. Sicurezza RSA + perché l'algoritmo base non è sicuro (x6)
2. RSA key key-generation (x6)
3. RSA scolastico (rsa plain): perché non si usa in pratica (x6)
4. algoritmo di generazione delle chiavi in RSA (x6)
5. RSA: algo generazione della chiave (x6)
6. RSA encryption and decryption performance improvements (x6)

Risposte:

Generazione chiavi

1. Si scelgono p e q due numeri primi, diversi e molto grandi (~ 200 cifre decimali)
2. Si calcola $n = p * q$ detto “modulo”.
3. Si calcola $\Phi(n) = (p - 1)(q - 1)$
4. Si sceglie randomicamente $e \in [2, \Phi(n) - 1]$ tale che $GCD(e, \Phi(n)) = 1$. Ossia e deve essere coprimo con $\Phi(n)$; poiché (esiste un teorema) se $GCD(e, \Phi(n)) = 1$ allora d **esiste ed è unico**.
5. Si calcola $d \in [2, \Phi(n) - 1]$ tale che $|e * d|_{\Phi(n)} = 1$. d è l'inverso di e in modulo $\Phi(n)$
6. p e q vengono eliminati.

Quindi le chiavi sono: $P_k = \langle e, n \rangle$ e $S_k = \langle d, n \rangle$

Complessità di ogni passo:

Calcolo di e

Si parte con $e = 0$:

1. $e++$
2. Si usa l'equazione di Diophantine: $GCD(e, \Phi(n)) = s * \Phi(n) + t * e$
3. Se $GCD(e, \Phi(n)) = 1 \rightarrow$ **fine**
4. Altrimenti si torna al punto (2)

Questo algoritmo ha complessità $\log(n)$

Calcolo di p e q

Per entrambi il procedimento è lo stesso:

1. $p = \text{rand}()$
2. p è pari $\rightarrow$ **torna al punto (1)**
3. p non è primo $\rightarrow$ **torna al punto (1)**
4. **Fine**

Il punto (3) è un test di primalità (*test di Miller-Rabin o il test di Fermat*) che richiede diverse operazioni, quindi per ottimizzare si mette il punto (2): si testano solo i numeri dispari.

Sia $P_i(x)$ il numero di numeri primi minori di x

Se $x \gg 1 \Rightarrow P_i(x) \sim \frac{x}{\ln(x)}$ e sono distribuiti in maniera uniforme in $[2, x - 1]$

La probabilità di trovare un numero primo in $[0, x] \Rightarrow \sim \frac{2}{\ln(x)}$; quindi mi aspetto $\sim \frac{\ln(x)}{2}$ tentativi

Operazioni in modulo

Supponiamo che n , a e b siano tutti su k bits.

$$|a + b|_n \sim O(k)$$

$$|a - b|_n \sim O(k)$$

$$|a * b|_n \sim O(k^2)$$

$$|\frac{a}{b}|_n \sim O(k^2)$$

$|a^{-1}|_n \sim O(k) \rightarrow$ Complessità del calcolo di d .

$|a^b|_n \sim O(k^3) \rightarrow$ Bisogna ottimizzarlo.

Square-and-multiply

Sia $[x_{k-1}, x_{k-2}, \dots, x_2, x_1, x_0]$ la rappresentazione in binario di x

$$\begin{aligned}|a^x|_n &= |a^{x_{k-1}2^{k-1}} * a^{x_{k-2}2^{k-2}} * \dots * a^{x_12^1} * a^{x_0}|_n = |(a^{x_{k-1}2^{k-2}} * a^{x_{k-2}2^{k-3}} * \dots * a^{x_12^0})^2 * a^{x_0}|_n \\ &= |((a^{x_{k-1}2^{k-3}} * a^{x_{k-2}2^{k-4}} * \dots)^2 * a^{x_12^0})^2 * a^{x_0}|_n = \dots = |(((a^{x_{k-1}})^2 * a^{x_{k-2}})^2 * \dots)^2 a^{x_0}|_n\end{aligned}$$

Se $x_i = 0$ allora $a^{x_i} = 1$ e quindi tolgo una esponenziazione alla 2° .

Operazioni Totali

$$\#MUL + \#SQ = \text{"Numero di bit a 1 in } x" + k$$

Supponendo un caso medio, in cui solo il 50% dei bit di x è a 1

$$\#MUL + \#SQ = \frac{k}{2} + k = 1536$$

Quindi con $k = 1024$ faccio 1536 moltiplicazioni su 1024 bit.

Altre ottimizzazioni

Possiamo usare un e piccolo, tanto è pubblico e ciò semplifica la cifratura, ma comunque mantenendo $x^e < n$, altrimenti $y = |x^e|_n = x^e$ e quindi $x = \sqrt[e]{y}$.

Invece un d piccolo è sconsigliato $\rightarrow$ Sempre avere $size(d) = size(n)$

Fast Decrypt

1. Calcolo $x_p = |x|_p$ e $x_q = |x|_q$
2. Calcolo $y_p = |x_p^{d_p}|_p$ con $d_p = |d|_{p-1}$ (precalcolato)
3. Calcolo $y_q = |x_q^{d_q}|_q$ con $d_q = |d|_{q-1}$ (precalcolato)
4. $y = |[q * C_p] * y_p + [p * C_q] * y_q|_n$

con $C_p = |q^{-1}|_p$ (precalcolato)

con $C_q = |p^{-1}|_q$ (precalcolato)

Ho un peggioramento della sicurezza: p e q non possono essere eliminati dopo la generazione delle chiavi. Se anche solo uno dei due venga svelato all'avversario, potrebbe fattorizzare n e trovare facilmente d .

Shannon

1. teorema di Shannon - Cifrario perfetto + dimostrazione (x5)
2. Shannon's definition of perfect cipher (x5)
3. teorema di Shannon, enunciato e dimostrazione (x5)

Risposta:

Definizione cifrario Perfetto:

$$\forall m, m' \in M, \forall c \in C, P_r [E_k(m) = c] = P_r [E_k(m') = c]$$

- Dati due messaggi possibili (quindi appartenenti allo spazio dei messaggi M).
- Dato un crittogramma possibile (quindi appartenente allo spazio dei crittogrammi C).

La probabilità che il crittogramma c sia ottenuto cifrando m o m' è la stessa. In parole povere: c non dà informazioni riguardo a m .

Teorema di Shannon

In un cifrario perfetto $|K| \geq |M|$.

1. Supponiamo per assurdo che ci sia un cifrario perfetto con $|K| < |M|$
2. Comunque, $|C| \geq |M|$ altrimenti il cifrario non sarebbe invertibile, poiché ad un crittogramma potrebbero corrispondere due o più messaggi.
3. Inoltre di sicuro $|K| < |C|$
4. Sia $m \in M$ tale che $P_r[M = m] \neq 0$ (un messaggio che può essere inviato nel canale).
5. Sia $c_i = E_{k_i}(m)$ con $k_i \in K$ (tutti i crittogrammi ottenibili cifrando m).
6. Dato il punto (3), $\exists c \in C, c \neq c_i \forall i$
7. Quindi $P_r[M = m | C = c] = 0 \neq P_r[M = m]$

Il punto (7) contraddice l'ipotesi di perfezione del cifrario: poiché c'è almeno un crittogramma c che non può essere ottenuto da m e quindi, vedendo c , posso escludere che è stato inviato m .

Quindi in un cifrario perfetto:

- La lunghezza in bit della chiave è maggiore o uguale alla lunghezza in bit del messaggio.
- Chiave composta da bit randomici.

Diffie-Hellman, Operazioni su Z_p^* e Man in the middle problem

1. Diffie-hellman + perché si lavora in un sottogruppo di Z_p^* (x4)
2. Protocollo Diffie-Hellman, come funziona? Si descriva il suo funzionamento e come stabilire i parametri necessari (x4)
3. Diffie-hellman (per scambio di chiavi) (x4)
4. Man-in-the-middle Attack in Diffie-hellman (x4)

Risposte:

Protocollo Diffie-Hellman

Protocollo rivoluzionario perché non prevedeva che A e B condividessero un segreto a priori, quindi non segreti a lungo termine. Poniamo le seguenti ipotesi:

- Sia p un numero primo grande (600 cifre, 2000 bits). Per generare p si usa lo stesso criterio che si usa per n in RSA.
- Sia $Z_p^* = [0, \dots, p-1]$ tali che $GCD(i, p) = 1$
- Sia $g \in Z_p^*$ detto "generatore" di un sottogruppo ciclico di Z_p^* tale che $1 < g < p$.
- p e g sono pubblici.

Allora il protocollo si articola:

1. Alice calcola la chiave segreta $a = rand()$
2. Bob calcola la chiave segreta $b = rand()$
3. Alice invia M1 a Bob:

$$M1 \quad A \rightarrow B: "Alice", y_A = |g^a|_p$$

4. Bob invia M2 ad Alice:

$$M2 \quad B \rightarrow A: "Bob", y_B = |g^b|_p$$

5. Alice calcola $K_{AB} = |y_B^a|_n = |g^{ab}|_n$

6. Bob calcola $K_{AB} = |y_A^b|_n = |g^{ab}|_n$

Proprietà di Z_p^* : perché si lavora in un sottogruppo di Z_p ?

Lavorare in un sottogruppo ciclico garantisce che le proprietà matematiche necessarie per la sicurezza e l'efficienza del protocollo siano soddisfatte.

Questo rende il protocollo resistente a diversi tipi di attacchi e più pratico da implementare.

- Consentono delle interessanti proprietà matematiche che aumentano l'efficienza computazionale.

$$|\log_g(\beta\gamma)|_p = |\log_g(\beta) + \log_g(\gamma)|_p$$

$$|\log_g(\beta^s)|_p = s * |\log_g(\beta)|_p$$

- Usare un sottogruppo ciclico riduce il rischio di alcuni tipi di attacchi, come quelli basati sulla fattorizzazione del modulo p (Pohlig-Hellman) o su proprietà aritmetiche specifiche del gruppo completo.
- In un sottogruppo ciclico, gli esponenti sono distribuiti uniformemente, il che significa che ogni possibile valore di g^k è ugualmente probabile. Questo contribuisce alla sicurezza del protocollo contro analisi statistica e attacchi basati su pattern ripetitivi.

Per avere la massima sicurezza è consigliabile lavorare un sottogruppo H grande e primo.

Diffie-Hellman: Attacco Man In the Middle

Articolazione dell'attacco:

1. Alice calcola la chiave segreta $a = rand()$
2. Bob calcola la chiave segreta $b = rand()$
3. L'avversario calcola $c = rand()$
4. Alice invia M1 a Bob:

$$M1 \quad A \rightarrow B: "Alice", y_A = |g^a|_p$$

5. L'avversario intercetta il messaggio e invia a Bob:

$$M1' \quad M \rightarrow B: "Alice", y_{AM} = |g^c|_p$$

6. Bob invia M2 ad Alice:

$$M2 \quad B \rightarrow A: "Bob", y_B = |g^b|_p$$

7. L'avversario intercetta il messaggio e invia a Alice:

$$M2 \quad B \rightarrow A: "Bob", y_{BM} = |g^c|_p$$

8. Alice calcola $K_{BM} = |y_{BM}^a|_n = |g^{cb}|_n$

9. Bob calcola $K_{AM} = |y_{AM}^b|_n = |g^{cb}|_n$

10. L'avversario ha sia K_{AM} che K_{BM} e si interpone nella comunicazione.

Questo attacco si evita utilizzando l'autenticazione e quindi i certificati.

Perfect Forward Secrecy, problemi e vantaggi

1. Perfect forward secrecy (Definizione + esempio di come si può ottenere) (x3)
2. Perfect forward secrecy (x3)
3. problema della perfect forward secrecy? (x3)

Hash functions: 3-proprietà, Merkle-Damgaard scheme, funzioni di compressione, guessing attack, birthday attack

1. Proprietà di una funzione Hash + attacchi + Merkle Damgard (Concetto generale + schema se si ricorda) (x7)
2. Funzioni hash (3 proprietà, attacchi alle varie proprietà) (x7)
3. Proprietà funzioni Hash, guessing attack (funzionamento e complessità) (x7)
4. Guessing attack (x7)
5. Birthday attack (paradosso del compleanno) (x7)
6. schema di Merkle-Damgard (hash più grandi da hash piccoli(?)), Come si realizza la funzione $h(\text{piccolo})$ (x7)
7. Definizione di funzione Hash sicura, quali proprietà deve possedere? Imposta e calcola la probabilità di un Birthday attack, con la complessità. Schema Merkle-Damgard. (x7)

Certificati: Informazioni minime, operazioni di verifica

1. Quali sono le informazioni minime che ci aspettiamo di trovare dentro un certificato? quali sono i controlli da fare prima di utilizzarlo? (x2)
2. Certificates (including OCSP, CRL and Cross certification) (x2)
3. When is a certificate considered valid? (x2)

DES, 2-DES e 3-DES

1. cifrario DES quando si usa? per quale motivo il 3DES è sicuro e la 2DES no? complessità di storage? complessità sui dati? trascurabile quella sui dati. se usassi 4DES? (x2)
2. DES, 2-DES e 3-DES: quanto conviene? (x2)

Square and multiply

1. $a^x \bmod n$ (multiply, generalizzazione) (x1)
2. parametro "e" in RSA (?) come si determina, a^x , square&multiply (esponente deve contenere il minor numero di 1-calcolo combinatorio(?)) (x1)

****MAC e HMAC****

1. MAC: costruzione HMAC (x1)

****DLP sulle curve ellittiche****

1. DLP non generici, $\mathbb{Z}_p^*$, vantaggio di passare alle curve ellittiche? (x1)

****Dictionary Attack e Rainbow-Table Attack****

1. Dictionary attack (è un kpa) (x1)
2. Da cosa dipende la dimensione del dizionario nel dictionary attack (da 2^n) (x1)

****RNG: proprietà per la sicurezza****

1. cifrari a caratteri: approssimazione block cipher, caratteristiche che deve avere un PRG per essere sicuro; LFSR: predicibile (x1)

****Varie****

1. Somma tra int (x1)
2. OS command injection (x1)
3. SQL Injection, quali sono le tecniche per contrastarlo? (x1)
4. Black box (x1)
5. Discuss about authenticated encryption schemes and argue about their security (x1)
6. Kerberos (x1)
7. Kerberos delegation (proxy and forwardable tickets) (x1)
8. Cifratura multipla (2E, 3DES): quanto conviene? (x1)
9. problema della canonizzazione dei nomi nel file system? limite whitelist e blacklist? (x1)
10. spiegare le vulnerabilità legate alle funzione di libreria, scanf e printf che hanno un numero di parametri variabili (x1)